# 不围棋游戏编程对抗赛

不围棋（NOGO）由围棋衍生而来，棋盘比围棋小，规则与围棋相反，玩起来比围棋简单。在本题中，你的任务是编写不围棋 AI 程序，即让程序自动下棋，并在北京大学 Botzone 平台（https://www.botzone.org.cn/）与其他同学对战；还将进行计算机学院、软件学院跨院 PK，决出两院总冠军。

## 一、游戏规则与术语：

1. 棋盘为 9×9 围棋棋盘。
2. 黑子先手，双方轮流落子，落子后棋子不可移动。
3. “连”的定义：“连”就是棋子和棋子接在一块，成为一个整体。横是连，竖是连，斜线不算连。例如图 1 中带方形的两颗棋子是相连的，带三角形的三颗棋子是相连的，右边的带圆形的三颗棋子不相连。
4. “气”的定义：“气”是围棋和不围棋里最核心的概念。一个棋子在棋盘上，与它直线紧邻的空点是这个棋子的“气”。棋子直线紧邻的点上，如果有同色棋子存在，则它们便相互连接成一个不可分割的整体，它们的气也应一并计算。棋子直线紧邻的点上，如果有异色棋子存在，这口气就不复存在。如所有的气均为对方所占据，便呈无气状态。例如图 2 中，角上一颗黑子有 2 口气，其右侧上边的一颗黑子有 3 口气，中间一颗黑子有 4 口气。右边两个白子相连有 6 口气，下边三个白子相连有 8 口气。

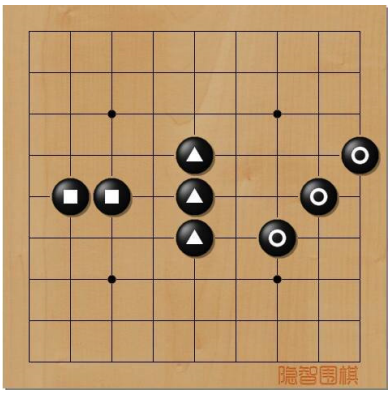


图 1 连

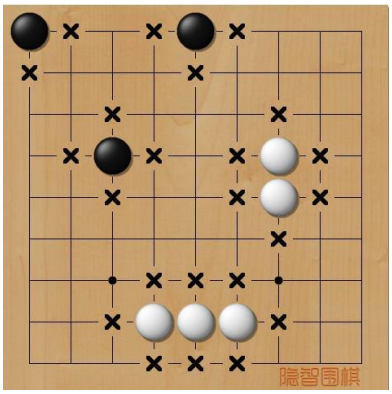


图 2 气

5. “吃子”的定义：落子后，使对方的棋子变成无气状态就是吃子。例如在图 3 中，如果白棋下在“×”标记点上，就会把黑棋变成无气，把黑棋吃掉。
6. 不围棋对弈的目标不是吃掉对方的棋子，恰恰相反，如果一方落子后吃掉了对方的棋子，则落子一方判负。
7. 如果一方在棋盘上某个点落子后，该子将呈现无气状态，那么相当于自杀，例如图 4 中，白棋若在“×”处落子即为自杀。不围棋对弈禁止自杀，落子自杀一方判负。
8. 对弈禁止空手（Pass），即轮到本方走棋时，若无棋可走，则本方判负。
9. 对弈结果只有胜负，没有和棋。
10. 每步走棋限时 1 秒，超时判负。内存限定 256MB。

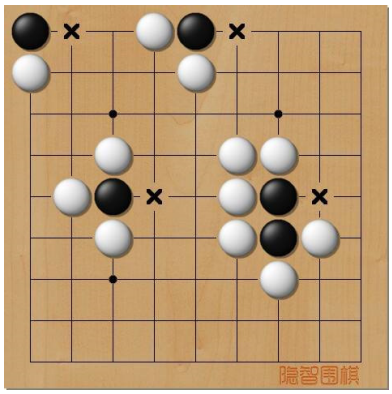


图 3 吃子

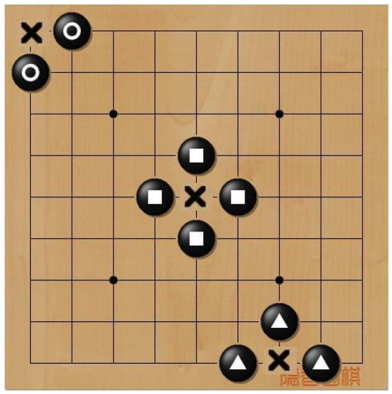

图 4 自杀

建议大家实际玩玩这款游戏，玩个十几局就掌握其规则及玩法了。PC 版已发给大家（若无法打开，可能需要安装 visual studio），手机版可在 app store 或安卓市场下载。大家也可以查阅相关网络资料，进一步了解不围棋的下棋技巧和经验。

## 二、代码实现：

大家无需掌握图形界面编程技术，只需将代码上传至北京大学Botzone对战平台（https://www.botzone.org.cn/），对战平台将自动进行图形化对战比赛，并给出排名。本题编程语言为C/C++语言，允许使用C++ STL。

### （1）棋盘坐标设定

9×9的二维棋盘分为横轴和纵轴两个维度，以左上角为坐标原点(0, 0)。坐标系如图5所示。在程序中棋盘信息存储在数组int board[9][9]中，下标从0开始，元素board[x][y]有0、1、-1三种可能取值，分别表示棋盘(x, y)处为空白（没有任何棋子）、对方棋子、我方棋子。

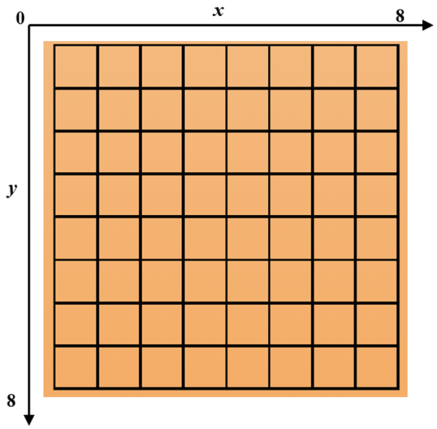


图 5 棋盘坐标系

### （2）你的程序与对战平台的交互方式

本题提供给大家样例程序，样例程序已经包含了程序与对战平台的交互功能，你只需在样例代码基础上开发，基于当前棋局，输出本回合的决策结果。你的程序（在botzone网站里也称为bot）的生命周期仅为一次决策：读入当前棋盘状态，输出本回合你的下棋（落子）位置。在整个对弈过程中，对战平台会多次调用你的程序（每下一步棋调用你一次，直至对弈结束）。

**了解下面标注黄色高亮的内容有助于理解程序的运行原理和流程。若不想或无法理解下面内容，可跳过而直接看（3）节，对完成本题没有影响。**

对战平台每次调用你的程序时：首先将之前对方和本方落子的位置作为输入传送给你的程序，从而建立起棋盘信息board数组，然后你的程序根据棋盘信息进行决策，并向对战平台输出你的决策结果，即本方的落子位置。

你的bot与对战平台的交互方式（简单交互）：你的程序通过一系列数字与对战平台进行交互，格式形如下表左侧所示。而你的程序就是要读取这些数字，建立棋盘信息，并基于棋盘信息做决策，最后输出第$k$回合本方落子位置。详见示例程序。例如图6的棋盘状态对应下表右侧的输入。

| | |
|---|---|
| $k$ //双方已下棋的回合数 | 4 |
| $x_1^{对方}$ $y_1^{对方}$ //第1回合对方落子位置 | -1 -1 |
| $x_1^{本方}$ $y_1^{本方}$ //第1回合本方落子位置 | 2 3 |
| $x_2^{对方}$ $y_2^{对方}$ //第2回合对方落子位置 | 0 8 |
| $x_2^{本方}$ $y_2^{本方}$ //第2回合本方落子位置 | 7 5 |
| …… | 6 8 |
| $x_{k-1}^{对方}$ $y_{k-1}^{对方}$ //第k-1回合对方落子位置 | 5 8 |
| $x_{k-1}^{本方}$ $y_{k-1}^{本方}$ //第k-1回合本方落子位置 | 1 4 |
| $x_k^{对方}$ $y_k^{对方}$ //第k回合对方落子位置 | |

假设本方是黑方，对于上面的信息，可知之前的棋应该是这么下的：我方第一回合收到(-1,-1)，然后我方下棋(2,3)，然后对方下棋(0,8)，然后我方下棋(7,5)…以此类推，形成图6的棋盘。

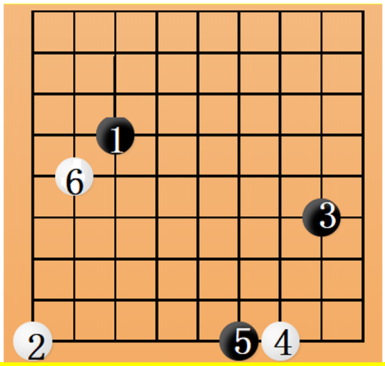


图6 对战平台与博弈程序通信示例

关于bot与平台的交互方式的更详细描述可参考：

https://wiki.botzone.org.cn/index.php?title=Bot#.E4.BA.A4.E4.BA.92

https://wiki.botzone.org.cn/index.php?title=NoGo

## （3）博弈程序编写

提供给大家JSON交互和简单交互的两个样例程序，分别为nogoai.cpp和nogo_simple.cpp，里面包含了详细注释，容易阅读。代码中棋局初始化和输入输出操作你都不用管，**你只需要实现代码中下图红框部分，用你的策略替换掉红框中的样例策略。new_x和new_y就是你的决策结果。**

```
    x = input["requests"][i]["x"].asInt(), y = input["requests"][i]["y"].asInt();//对方, 注意此处
    if (x != -1) board[x][y] = 1;
    x = input["responses"][i]["x"].asInt(), y = input["responses"][i]["y"].asInt();//我方, 注意
    if (x != -1) board[x][y] = -1;
}
x = input["requests"][turnID]["x"].asInt(), y = input["requests"][turnID]["y"].asInt();//对方,
if (x != -1) board[x][y] = 1;
//此时board[][]里存储的就是当前棋盘的所有棋子信息,x和y存的是对方最近一步下的棋

/*******************************************************************************/
/***********在下面填充你的代码，决策结果（本方将落子的位置）存入new_x和new_y中****************/
    //下面仅为随机策略的示例代码，可删除
int available_list[81]; //合法位置表
int k = 0;
for (int i = 0; i<9; i++)
    for (int j = 0; j<9; j++)
        if (judgeAvailable(i, j, x == -1 ? 1 : -1))
        {
            available_list[k] = i * 9 + j;
            k++;
        }
int result = available_list[rand() % k];
int new_x = result / 9;     int new_y = result % 9;
/***********在上方填充你的代码，决策结果（本方将落子的位置）存入new_x和new_y中****************/
/*******************************************************************************/

// 输出决策JSON
Json::Value ret;
Json::Value action;
action["x"] = new_x; action["y"] = new_y;
```

使用任何 IDE 均可运行样例程序。

## （4）博弈程序的调试

### ➢ 本地调试

在本地编译运行程序，本地调试时可采用“简单交互”方式，并结合输入输出重定向，以数字形式作为输入，并观察程序输出结果。

### ➢ 在Botzone网站上在线调试

Botzone 提供了比较完善的调试功能。点击网页右上角的“我的 Bot”，然后点击下方“创建新 Bot”，如图 7。然后填写 bot 信息，上传代码文件，如图 8，。请不要勾选“开源”，为避免抄袭，将对开源者扣分。**Botzone 提供了较好的版本管理功能，方便管理程序的各个版本。**若想对已有的 bot 增加新的版本，可如图 9 所示点击加号。

图 7 创建新 Bot

图 9 为已有 bot 增加新版本

图 8 配置 Bot 信息上传代码

图 10 创建游戏桌

在线调试，可回到首页，手动创建游戏桌，如图 10 所示。然后指定玩家进行对局，如图 11 所示，对局双方可以是已有的 Bot，也可以是房主（人类），即你自己。

图 11 指定玩家进行对局调试

在对局页面（也可通过右上角菜单“我的对局”选择对局点击“回放”）在线调试，点击下方“切换到调试模式”，如图 12 所示。详细调试信息可以点击右下角“log 查看工具（调试用）” 查看每回合每方具体的输入和输出信息，如图 13，选择“查看完整 Log”，也可以查看程序的运行时间。为防止程序超时，可以尝试进行卡时，比如在每次迭代/循环时，通过 clock()或者 time()函数来判断程序运行了多久，从而决定是否提前返回。

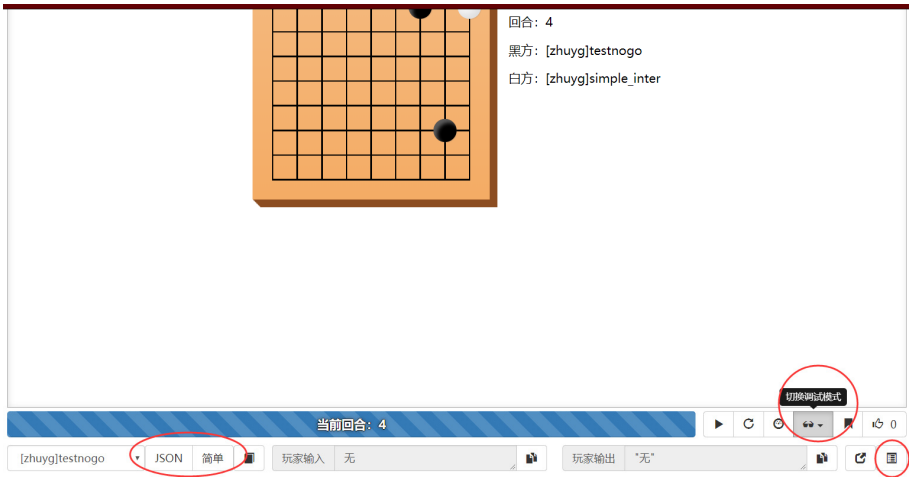


图 12

图 13

**额外提示 1：你的程序在本地的输出和平台的输出不一样怎么办？**（1）你的程序可能存在漏洞，在平台上可能会暴露出数组越界、地址越界、除零、未初始化等问题。建议仔细检查自己的代码。TScanCode 是一个很好的检查代码漏洞的工具。（2）如果采用了随机数，可能你本地生成的随机数和平台生成的随机数不一致。

**额外提示 2：关于超时和卡时**

在比赛的时候，bot 程序会被 Botzone 调度到一个评测机集群上运行，评测机是两台物理机上的多个虚拟机，当平台负载比较高的时候，物理机会存在总线竞争之类的问题，导致虚拟机内的运行速度变慢，比如原本 bot 一步决策花 0.5s，可能平台高负载的时候就要花 0.9s。可能导致**个别同学平时自己创建游戏桌单局对弈时都不会超时，但在比赛中就被判超时**。

因此建议大家在 bot 程序中加入卡时的代码，比如程序中有个耗时循环（如 MCTS 迭代次数）的话，不要固定循环次数，而是在每次循环的时候通过 clock()函数获取当前程序已运行的时间，当程序运行接近 1 秒时就退出循环，这样即使运行速度变慢，也不会超时，只是循环次数变少了而已。例如：

```
int threshold = 0.95 * (double)CLOCKS_PER_SEC; //CLOCKS_PER_SEC 和 clock()的含义可百度
int start_time, current_time;
start_time = current_time = clock();
while (current_time - start_time < threshold){ //到 0.95 秒立即跳出循环
    monte carlo tree search……
    current_time = clock();
}
```